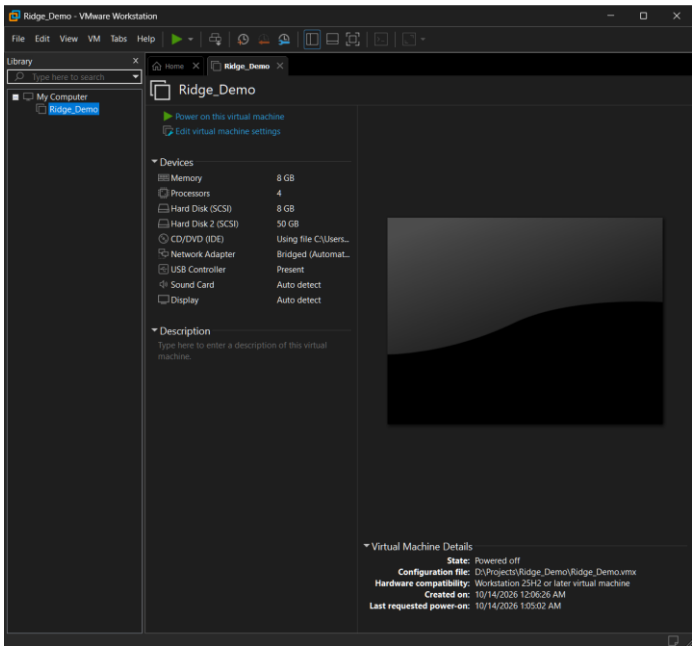
Junway Lin

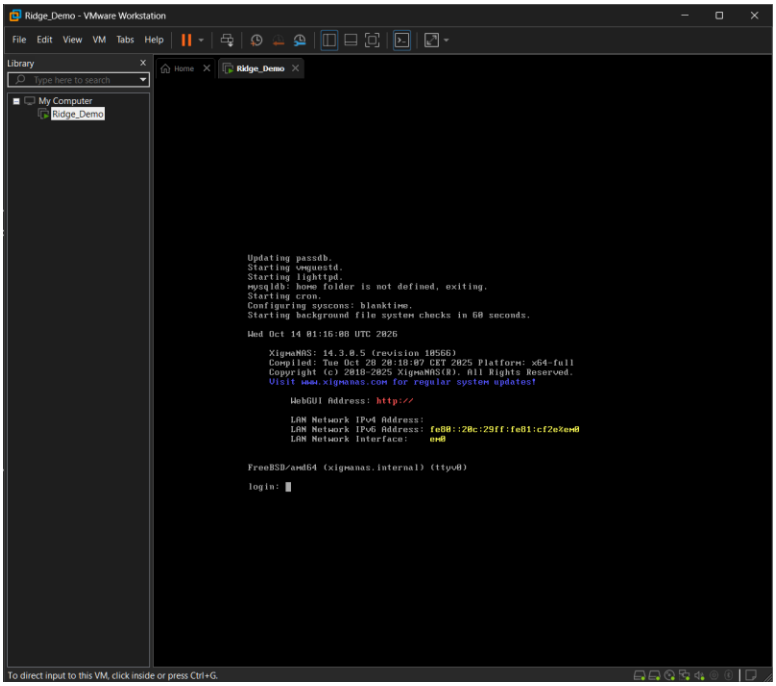
Question 1

1.1 Downloaded XigmaNAS image from Github Repo.

1.2 Created a virtual machine and installed XigmaNAS into the virtual machine.



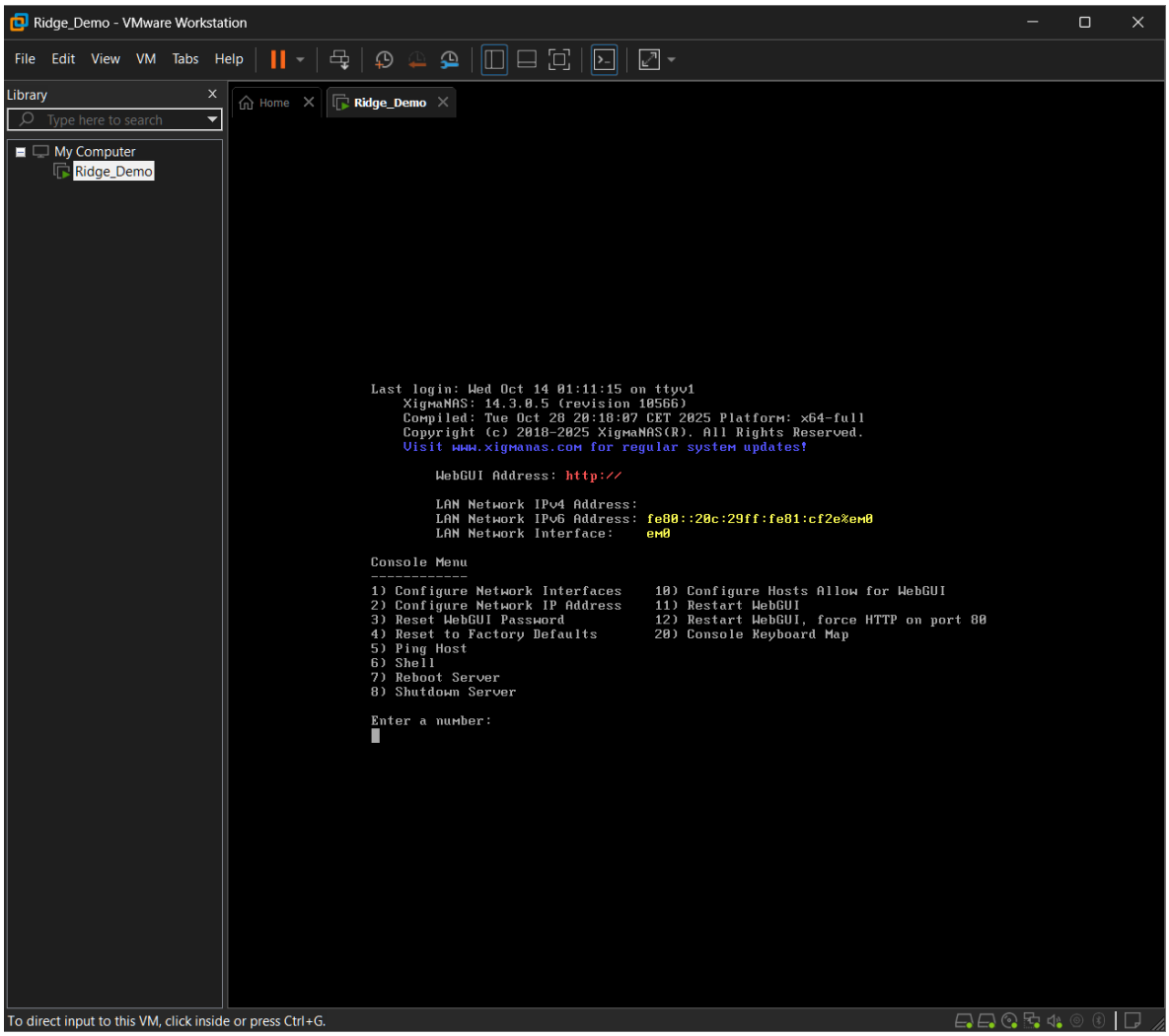
Complete installation



Intial startup (login: admin, password: xigmanas; by default)

Junway Lin

Since this is not being pushed into production, I will keep it simple and leave the login details as the default.



Could not proceed due to lack of LAN cable and the order would not ship on time, hence, I used a workaround. I set up my mobile device as a host for a LAN and connecting to my computer, ensuring to enable other users to connect to the shared network so that VMware can read the IP.

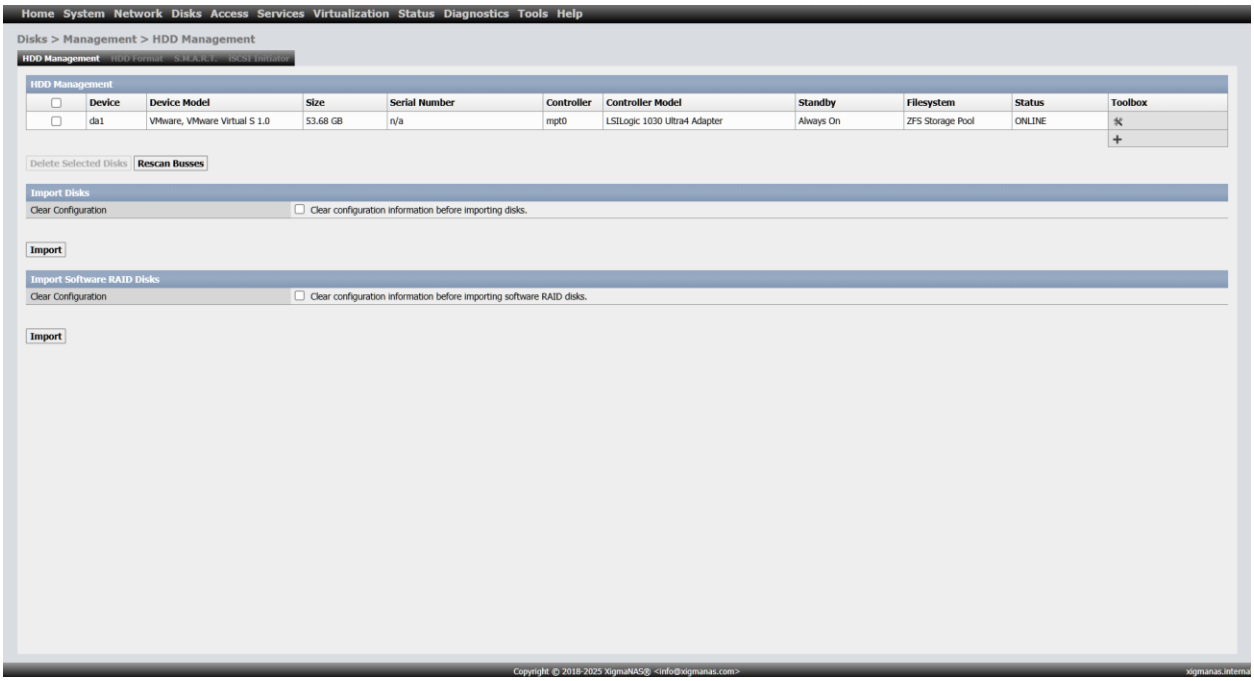
**Did not include the IP screenshot here for privacy purposes.

Shutdown can be done here by entering 8 and enter.

Junway Lin

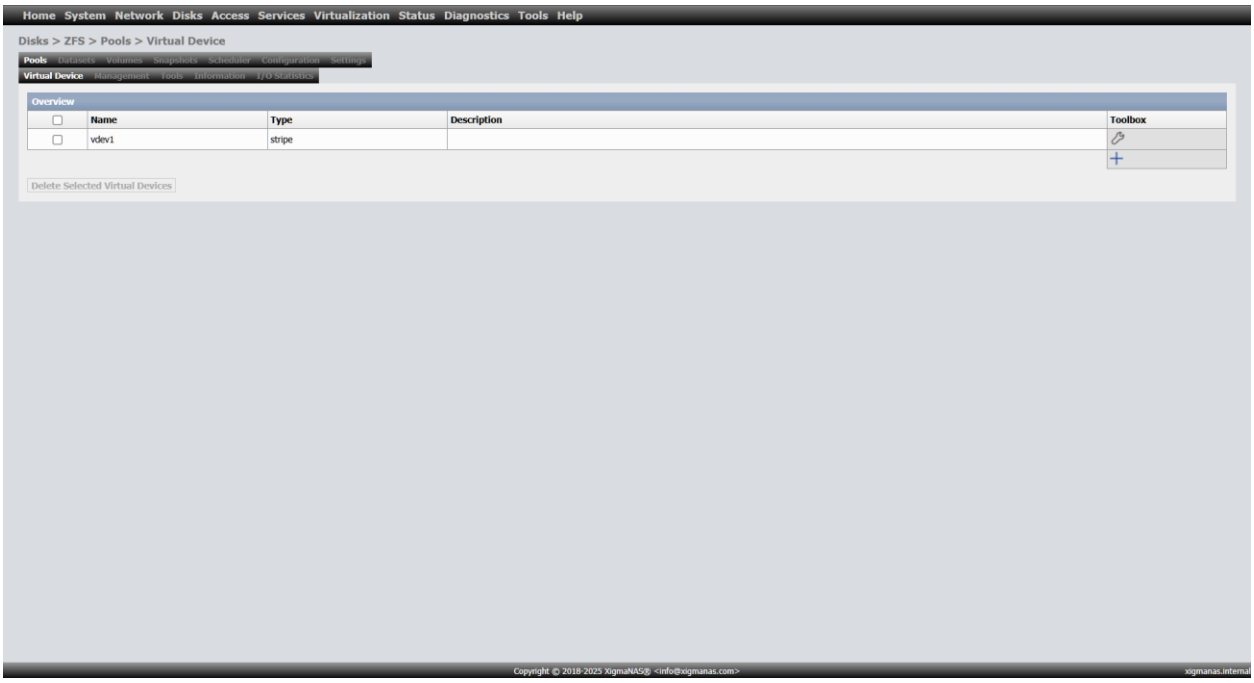


Normal startup and accessing the WebUI

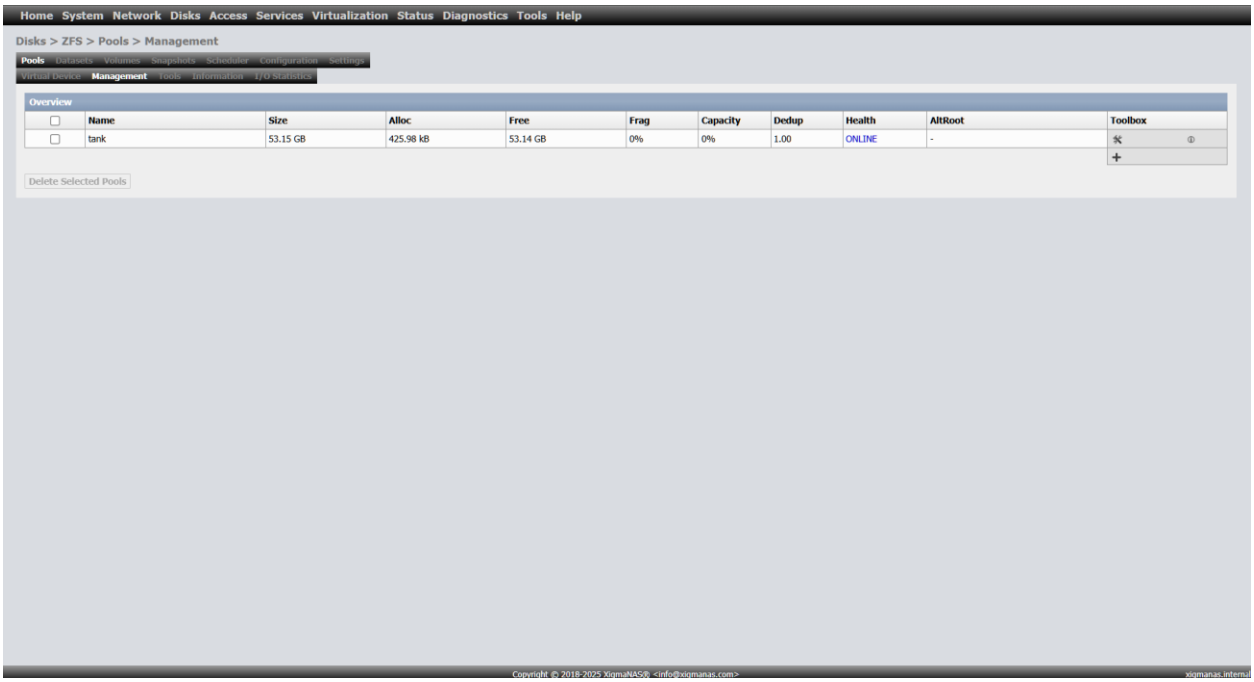


Imported and formatted storage devices (added the 50gb disk allocated at the initialization of workstation, while leaving out the 8gb disk which is our dedicated boot disk)

Junway Lin

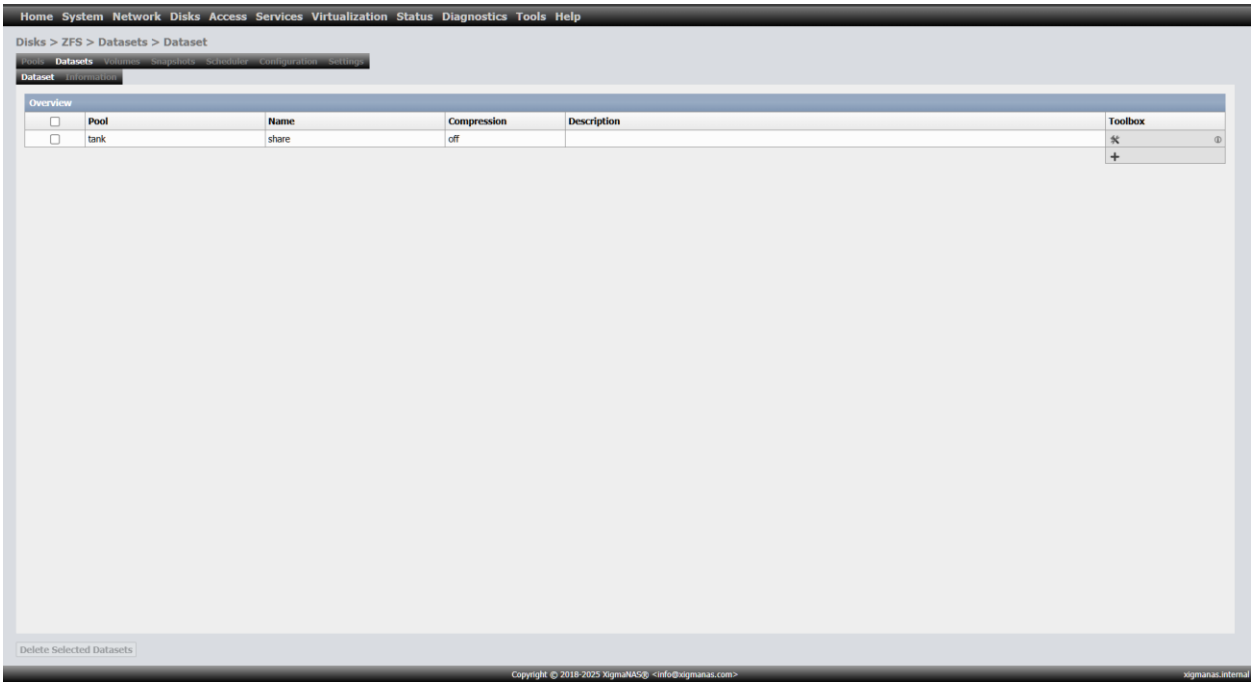


Initialized virtual device

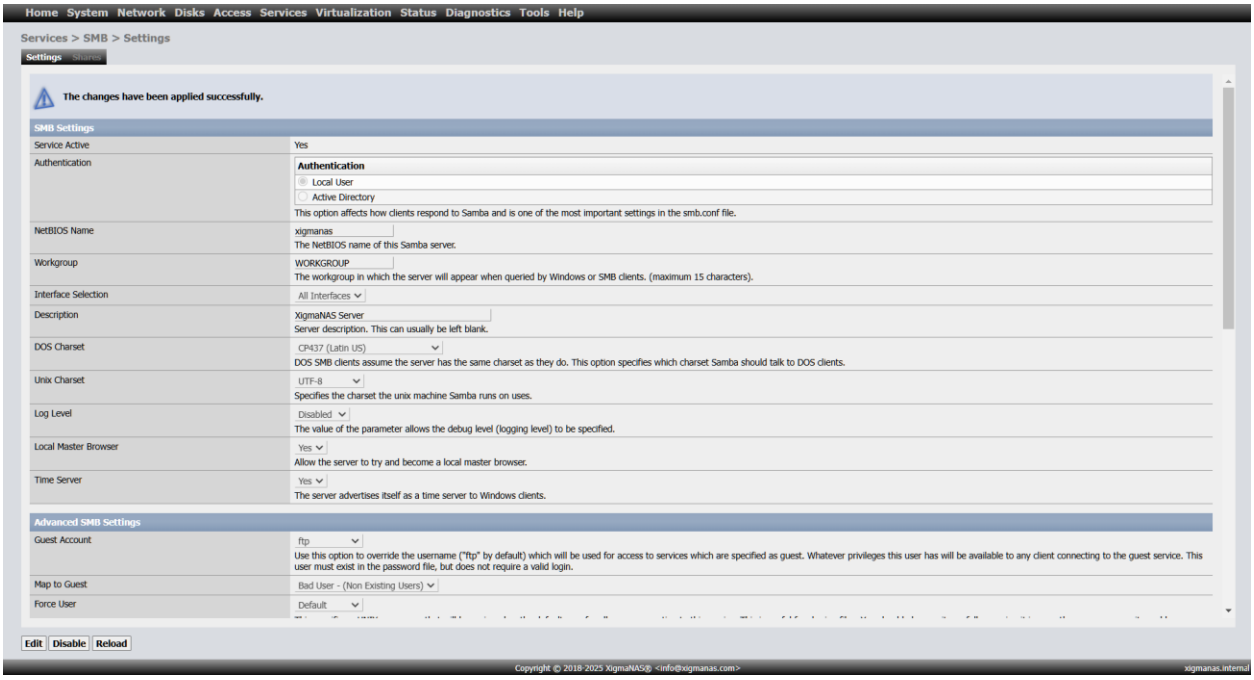


Initialized pool

Junway Lin

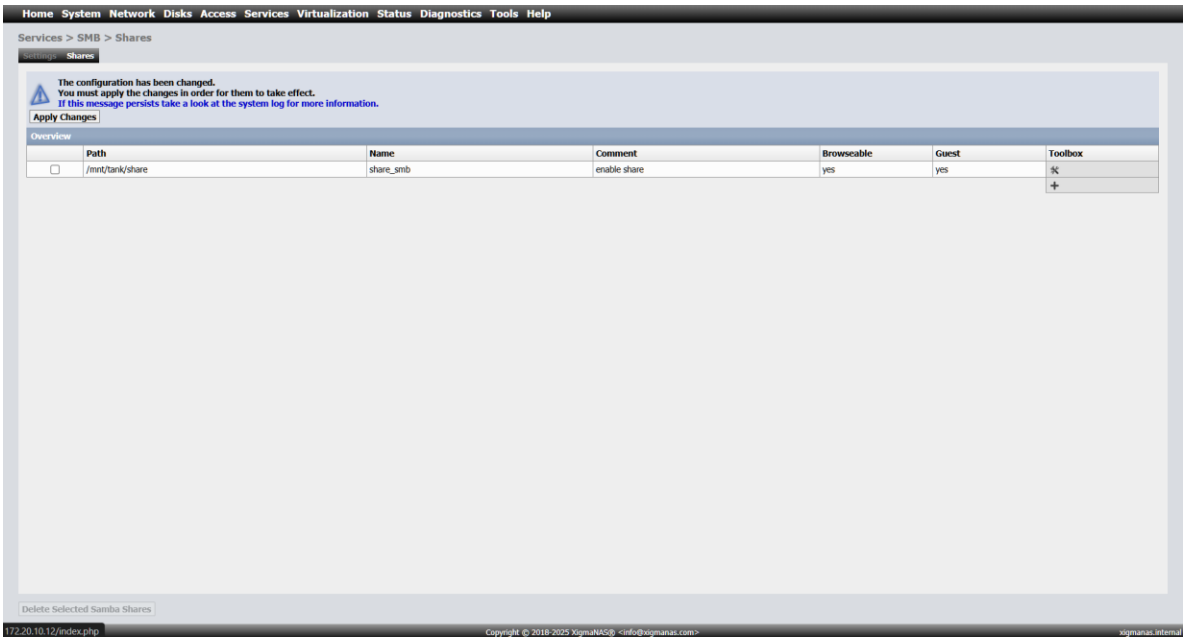


Initialized dataset

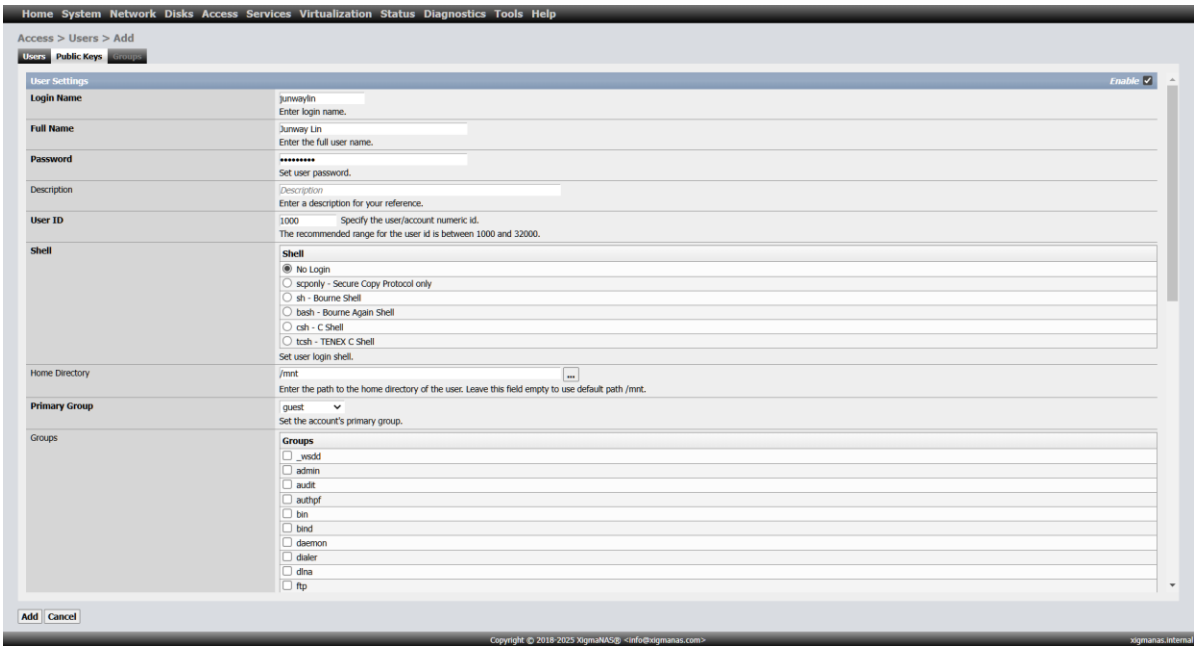


Enabled SMB service

Junway Lin



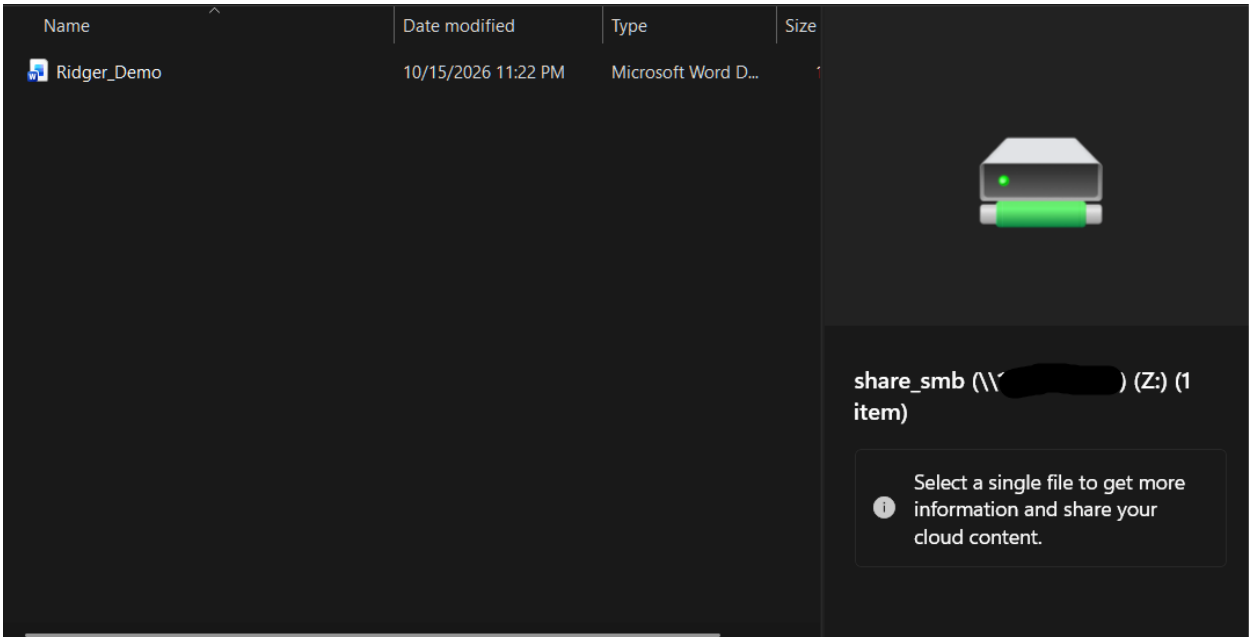
Added share to services



Create user with guest access

A single-disk stripe has **no redundancy**. In real deployment it is standard to use a mirror or RAID-Z. It is still possible to demonstrate datasets, snapshots, and SMB access with this setup.

Junway Lin



Mounted the share from host machine and copied a file in to test. Then shut down server and rebooted to confirm that the file persists.

1.3 Explaining the principles, functions, advantages and disadvantages of XigmaNAS:

XigmaNAS is essentially a FreeBSD-based embedded NAS (Network Attached Storage) distribution that wraps FreeBSD's storage capabilities in a web-based management interface, turning it into a plug-and-play NAS system.

Core design philosophy:

- Embedded-first: The system loads into RAM and only writes to the boot medium when configuration changes occur. This provides tamper resistance and extends the lifespan of USB/CF flash media
- Storage-focused: It doesn't chase flashy GUIs; the priority is stable, efficient file-sharing services

Key components:

- Underlying OS: FreeBSD (provides hardware drivers, network stack, ZFS filesystem)
- Management interface: PHP-based WebGUI
- Core storage: ZFS (recommended), software RAID, UFS

Junway Lin

ZFS (Zettabyte File System) is an advanced file system and volume manager developed by Sun Microsystems. Unlike traditional file systems (like NTFS or FAT32) that just organize files, ZFS combines the file system and the volume manager into one. Its core superpowers are:

- Data Integrity: Every piece of data has a checksum. If a disk silently corrupts a file (bit rot), ZFS detects it and automatically repairs it using redundant copies.
- Snapshots & Replication: It can instantly capture the state of the entire file system at a specific moment. You can send these snapshots to another server as a backup.
- RAID-Z: It has built-in software RAID that is much more resilient than traditional hardware RAID. It doesn't suffer from the "write hole" bug that can destroy data during a power outage.
- Pools: Instead of partitioning individual drives, you throw all your drives into a giant "pool" of storage.

Analogy: XigmaNAS is like a "Customizable Industrial Storage Warehouse"

- FreeBSD (The Operating System) is the concrete foundation and steel framework of the warehouse. It's incredibly strong and stable, but it's bare and hard to use.
- XigmaNAS (The NAS OS) is the management office, the loading docks, and the warehouse software built on top of that foundation. It provides a nice WebGUI so forklifts (users) can easily drop off and pick up boxes (files) via different doors (SMB, NFS, FTP).
- ZFS (The File System) is the ultra-secure, automated shelving system inside the warehouse. It knows exactly where every box is, checks the contents of every box every time it's moved, and can instantly take a photograph (snapshot) of the entire warehouse before a big delivery.

Main Functions

FreeBSD kernel, ZFS/UFS file systems, and daemons such as Samba, NFS, FTP and rsync. The WebGUI is PHP and writes configuration to a single config.xml. On apply, it generates the service config files (smb.conf, exports, and so on) and restarts the services. It inherits the m0n0wall-style architecture. The embedded mode loads the OS into RAM, so the boot device sees few writes.

- Storage management: ZFS (RAID-Z/RAIDZ-2/RAIDZ-3), software RAID 0/1/5, JBOD, UFS, disk encryption, S.M.A.R.T. monitoring with email alerts

Junway Lin

- Network protocols: SMB/CIFS (Samba), NFS, AFP (Apple Filing Protocol), FTP/SFTP, iSCSI (initiator and target), rsync, Unison, TFTP, BitTorrent, and more
- Additional services: Built-in Syncthing (file sync), Transmission (BitTorrent client), Lighttpd (web server), MariaDB + phpMyAdmin, VirtualBox host (managed via phpVirtualBox)
- Network features: 802.1q VLAN, link aggregation, CARP (high availability), HAST (high-availability storage), Wake On LAN
- Monitoring and alerts: S.M.A.R.T., SNMP, Syslog, UPS support (NUT), email notifications

Advantages

1. Very low hardware requirements: The embedded installation needs as little as 512MB RAM and a 2GB boot drive. User reports confirm it runs smoothly on an HP microserver with only 4GB RAM, meaning you can repurpose old hardware instead of buying new equipment.
2. Native ZFS support: ZFS is integrated at the core level, not bolted on. You get data integrity checksums, snapshots, replication, compression, deduplication, and RAID-Z without needing a separate storage layer.
3. Free and open source: No licensing fees, no subscription tiers, no feature gating. Full source code access (which is why your assignment involves modifying and recompiling it).
4. Rich protocol coverage: Out of the box it speaks SMB/CIFS, NFS, AFP, iSCSI, FTP, rsync, and more, covering Windows, macOS, Linux, and virtualization clients without extra packages.
5. Embedded mode extends flash lifespan: Since the OS runs from RAM, USB sticks and CF cards aren't constantly written to, avoiding the premature failure common in other NAS distros.
6. Active community and documentation: The project has been maintained continuously since 2005, with a dedicated forum, wiki, and regular release cycle.

Disadvantages

1. Steep learning curve: The WebGUI exposes a lot of low-level options (ZFS tuning, iSCSI target configuration, etc.). Beginners used to Synology's polished DSM or QNAP's QTS will find it less handholding.

2. Dated user interface: The WebGUI is functional but visually dated compared to commercial NAS systems. It prioritizes control over aesthetics.
3. Smaller plugin ecosystem: Compared to TrueNAS SCALE (with its app catalog) or OpenMediaVault (with Docker/Portainer integration), XigmaNAS has fewer one-click apps. Adding services often means manual configuration.
4. Documentation can lag behind releases: Some features added in recent versions aren't fully documented in the wiki. You may need to rely on forum posts or source code reading.
5. No official commercial support: Unlike TrueNAS (iXsystems) or Unraid (Lime Technology), there's no paid support tier. You depend on the community for help.
6. ZFS RAM requirements: While the base system runs in 512MB, using ZFS properly (especially with deduplication) can demand 8GB+ RAM. The low minimum spec is somewhat misleading if you plan to use advanced features.
7. My own opinion: it forces the use of VMware, which is not a friendly work environment (based on current and past experiences).

Question 2

2.1. Log in to GitHub or other available websites to download the source code of XigmaNAS; https://archive.freebsd.org/old-releases/amd64/amd64/ISO-IMAGES/14.3/

During installation of freebsd, there was an issue creating the partition manually. Thus, I worked around it using shell commands:

gpart create -s GPT da0

gpart show # check that the partition was created

gpart add -t freebsd-boot -s 512K da0

gpart add -t freebsd-swap -s 4G da0

gpart add -t freebsd-ufs da0

newfs -U /dev/da0p3

mkdir -p /mnt

mount /dev/da0p3 /mnt

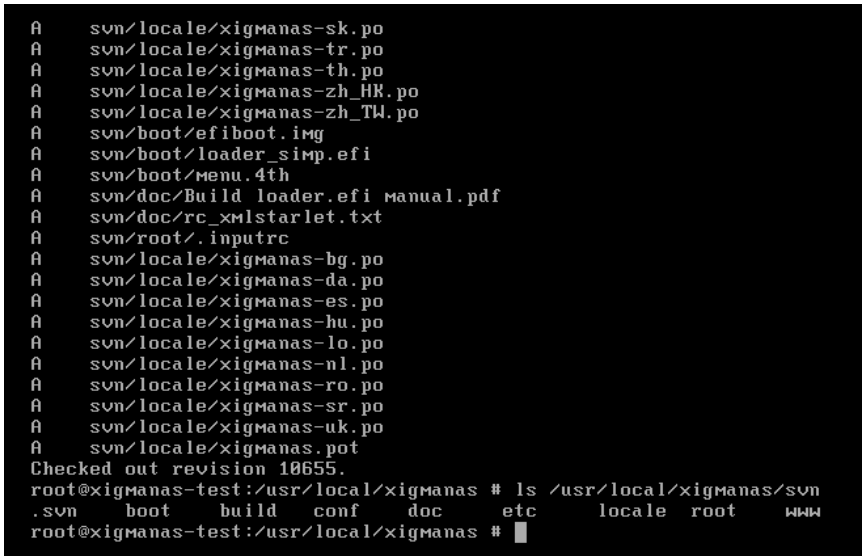
gpart bootcode -b /boot/pmbr -p /boot/gptboot -I 1 da0

echo '/dev/da0p3 / ufs rw 1 1' > /tmp/bsdinstall_etc/fstab

Junway Lin

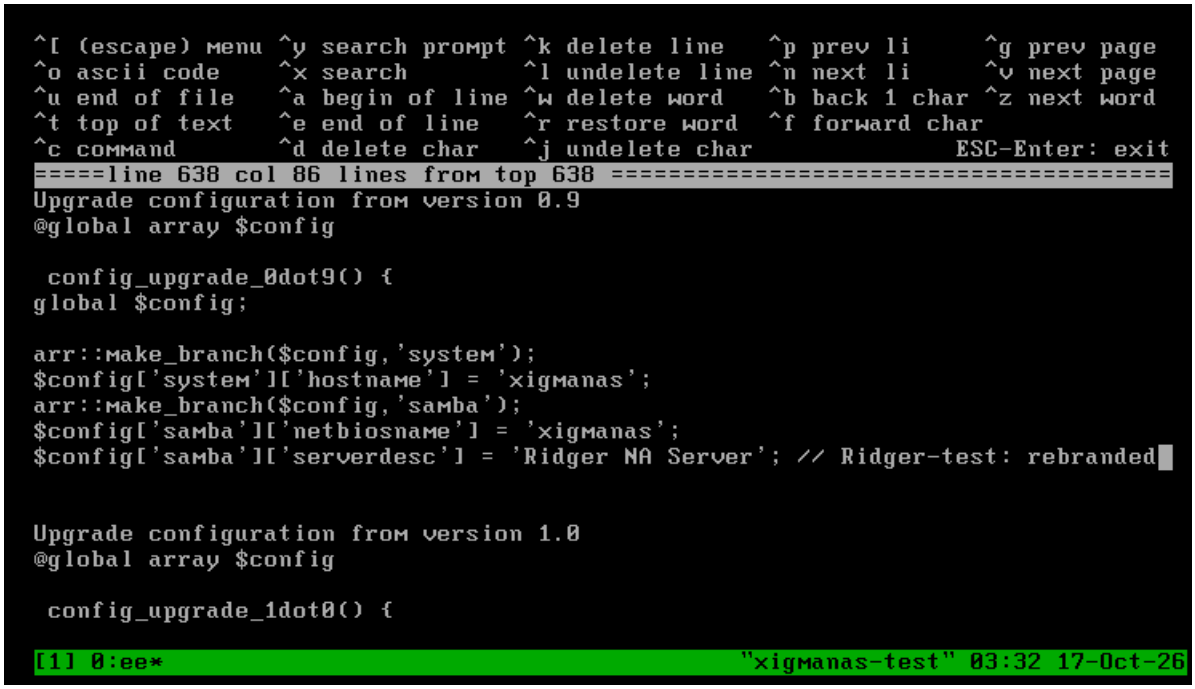
login: root

password: xigmanas



Fetched the source code

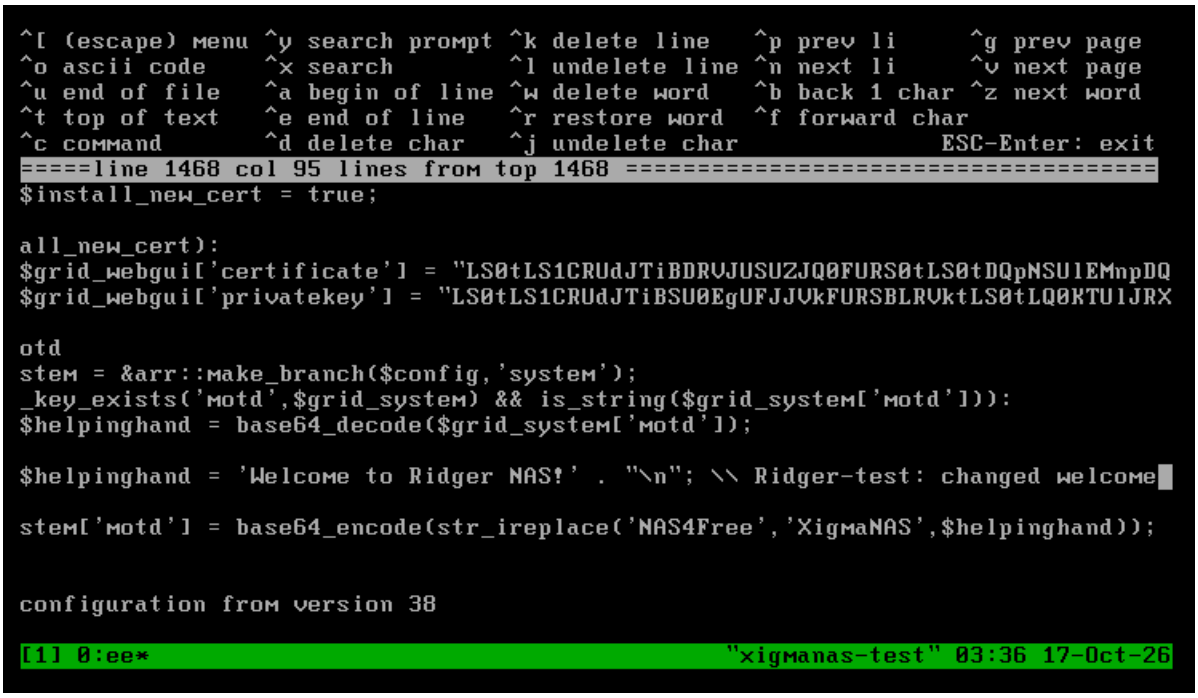
2.2. Modify or annotate the relevant code such as brand Logo, Icon and key strings;



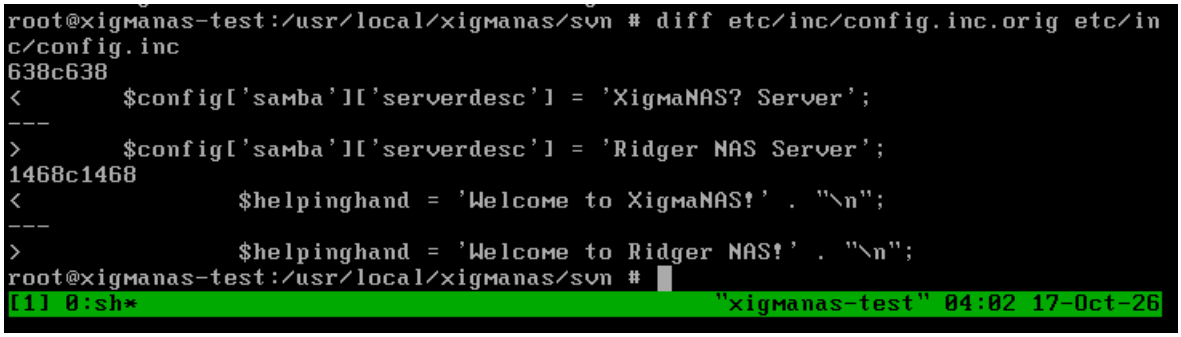
Modifying server description (Change ‘XigmaNAS Server’ to ‘Ridger NAS Server’)

***Later realized there was a typo during comparison and resolved before compiling.

Junway Lin



Modify welcome message (Change ‘Welcome to XigmaNAS!’ to ‘Welcome to Ridger NAS!’)



Check the changes.

Shortcut to changing them since we now know the exact lines:

cd /usr/local/xigmanas/svn

cp etc/inc/config.inc.orig etc/inc/config.inc

LC_ALL=C sed -I ‘’ “638s/= ‘.*Server’/= ‘Ridger NAS Server’/” etc/inc/config.inc

LC_ALL=C sed -I ‘’ “1468s/Welcome to XigmaNAS!/Welcome to Ridger NAS!/” etc/inc/config.inc

diff etc/inc/config.inc.orig etc/inc/config.inc

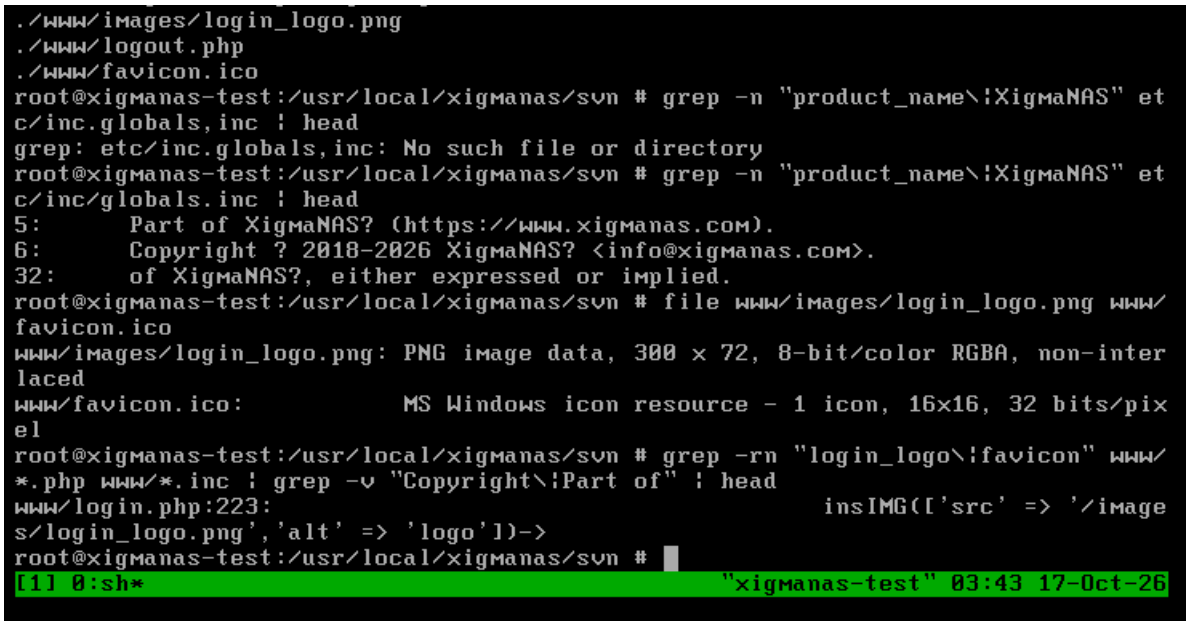


Image to replace (login logo is size 300x72, favicon icon is size 16x16)

2.3. Recompile into an installation file or image;

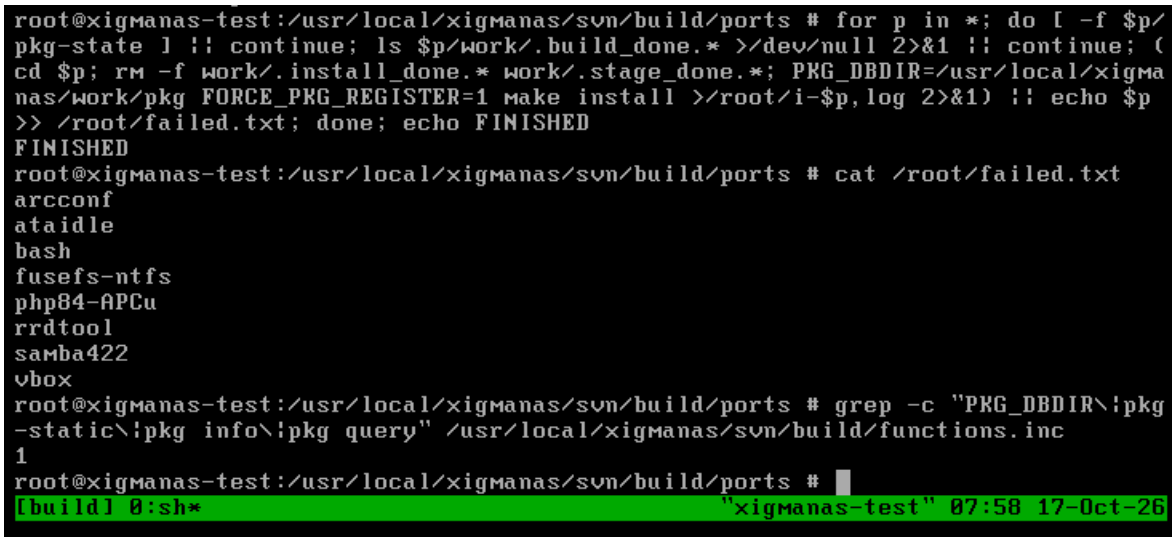
I attempted to build the file (with changes). However, when I got to step 6 (building the ports), I encountered multiple errors (version error, compability issues, cannot find the arcconf file, etc.). Eventually, after a couple hours, I was able to get it to start building the ports (this part was extremely long and required the longest amount of time, waiting for it to complete).

After 4 hours of compiling, it encountered yet another error this time building port: samba422. FAM issue across multiple files (needed it to be set to OFF)

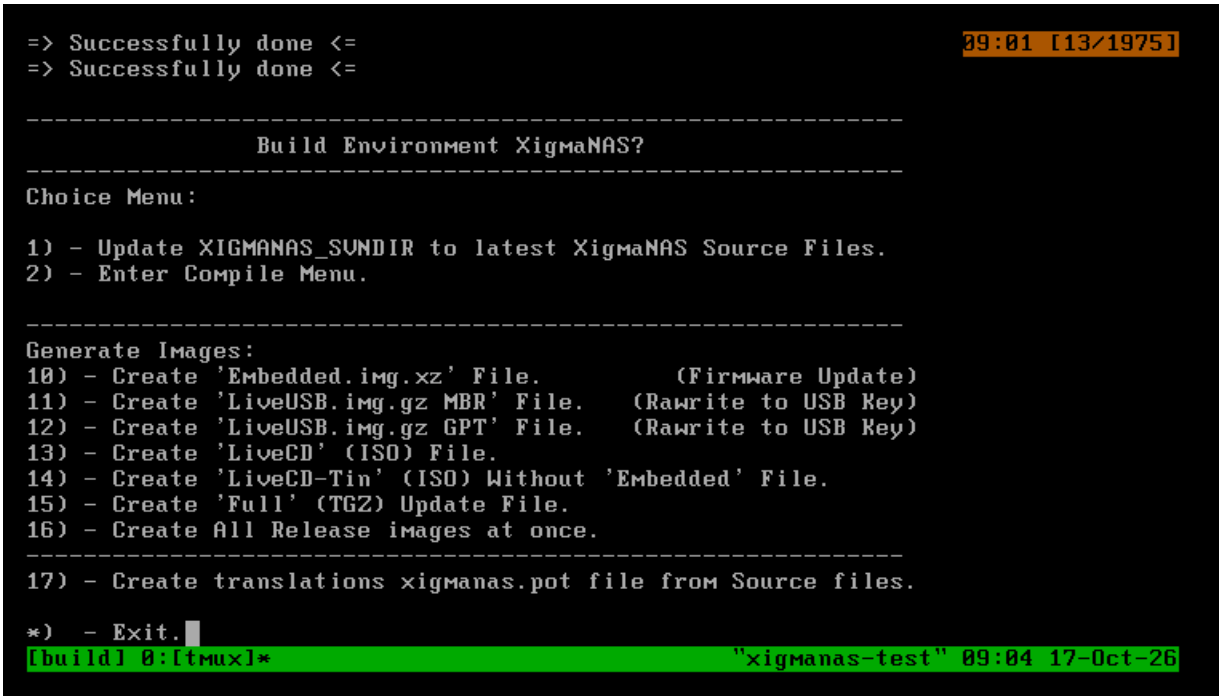
Another issue, when testing compiling step 6, there was yet another issue. This time, however, was due to the makefile compiling rust which is far too resource intensive, causing it to force exit. I worked around it by skipping building the Rust compiler on my machine. Needed a ports tree because the script expects 1.99.1 but the installed package is 1.98.1. Additonally, as an additional safeguard, I upgraded my allocated Vmware RAM (8->16gb) to prevent future bottlenecks.

Junway Lin

Now a new issue occurred with the syncthing port (version mismatch) and rsync (which raised vulnerabilities). Now there is an issue with installing the the bash port.



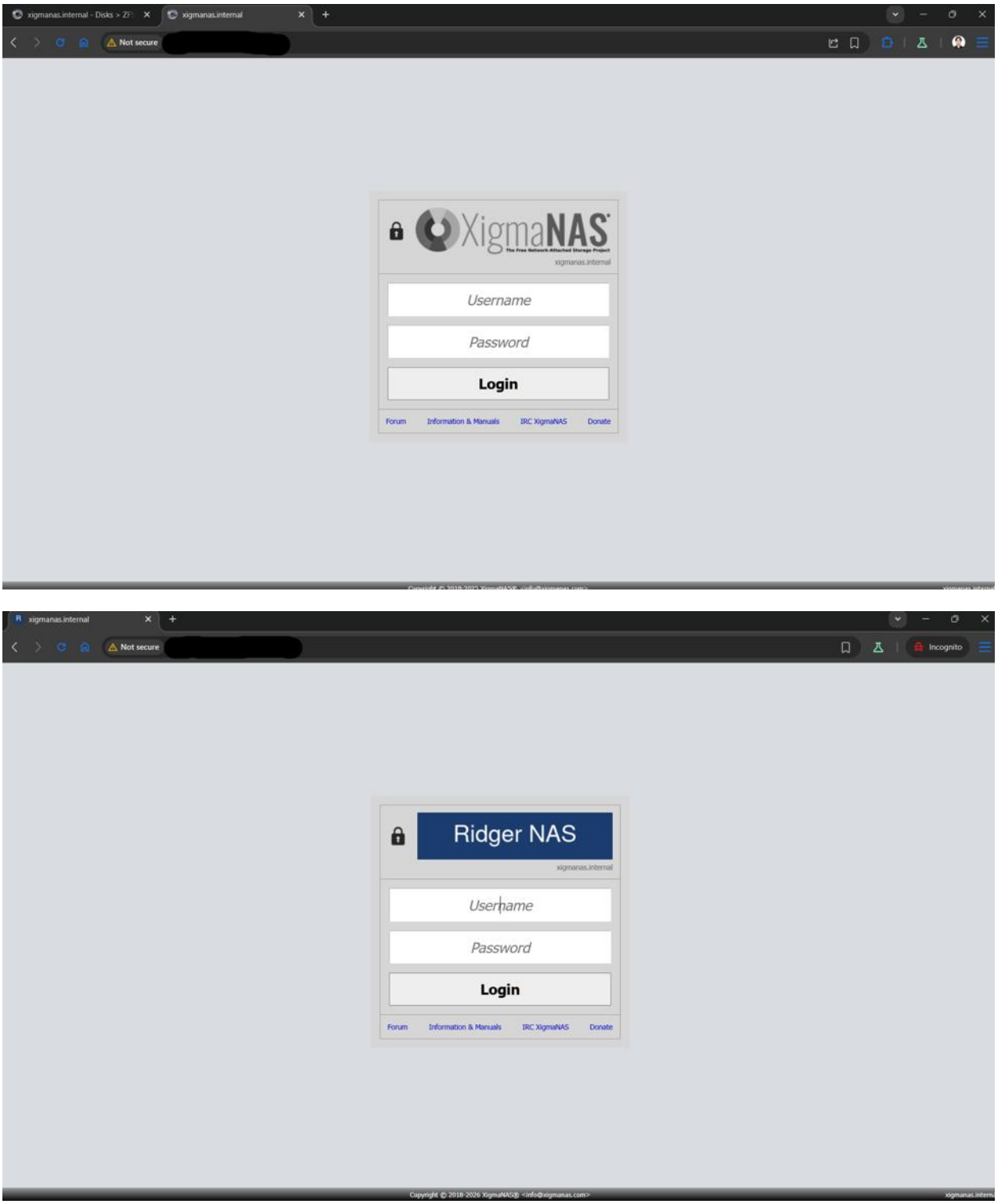
These ports failed to install. For the purposes of the demo, the most important is samba422 and the rest can be ignored.



Completed recompilation

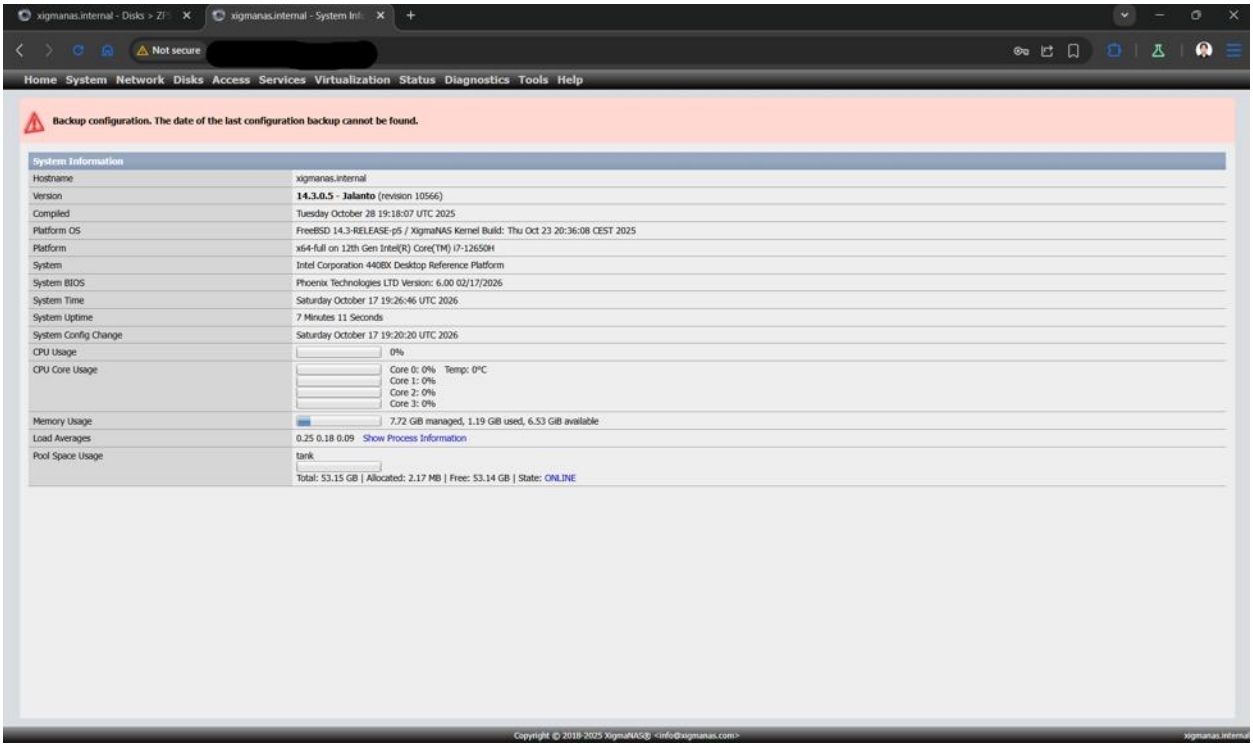
Junway Lin

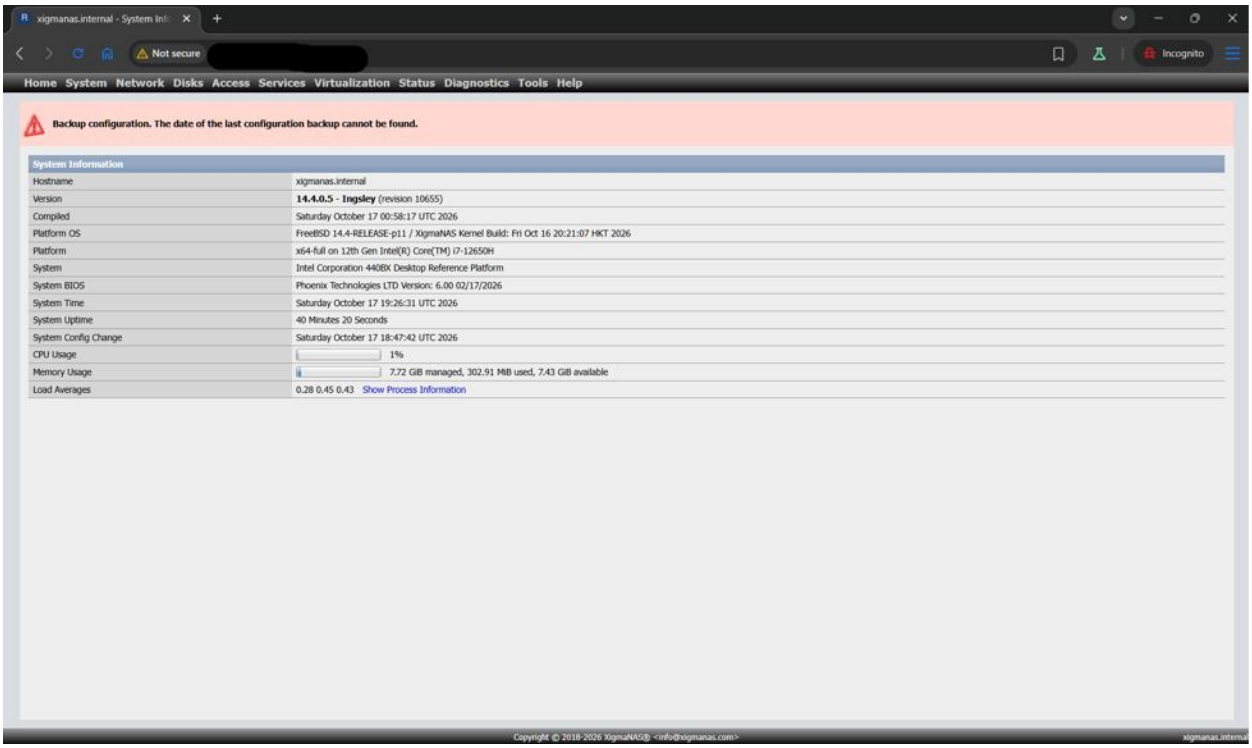
2.4. Additionally create another virtual machine, and according to your modified and compiled version, compare it with the original version to observe the differences between the two;



Comparison of Login Pages (Greatest Difference in Change)

Junway Lin





Comparison of home dashboard

Junway Lin

The new login page has the “Ridger NAS” banner where the XigmaNAS logo was, so the logo change worked. The favicon changed to the “R” too. The one thing that didn’t change is the tab title, which is just the hostname (xigmanas.internal), not a branded string. The welcome message is a different matter. I assumed it was a login banner, but that was a guess, and the dashboard clearly doesn’t show it. The string at config.inc line 1468 is probably a default value that only appears somewhere specific, such as a banner or message setting.

2.5. Spend 10 minutes explaining your work process;

1. Setup and source. I built on a FreeBSD VM, upgraded from 14.3 to 14.4 because the 14.3 ports collection was end of life. I checked out XigmaNAS trunk (revision 10655) with svn and ran the official build/make.sh menu.

2. Rebranding. I changed the Samba description and the welcome text in config.inc, and replaced login_logo.png and favicon.ico. I left the license and copyright headers alone, as the BSD license requires.

3. Fixing the ports build (the bulk of the work). Each fix was a mismatch between trunk and the 2026 ports tree:

- Empty /usr/ports, fixed by cloning it, then switching to the 2026Q4 branch.
- Rust build killed by out-of-memory, fixed with more RAM, swap and a pre-built Rust package.
- Removed fam from the Samba options and made Samba’s Makefile python3.12 compatible.
- Relaxed the syncthing Go version pin, turned off docs for bash, and disabled vulnerability checks for the throwaway VM.
- Stopped make clean from wiping finished ports with the rebuild option and NOCLEANDEPENDS.

4. Install into the image. My hand-typed install loop installed about 70 ports onto the build VM instead of rootfs. I switched to the menu’s own install and made its loop tolerate failures, a one-line edit to functions.inc. Samba and the other essentials landed; bash registration, rrdtool, arcconf and vbox did not, which was acceptable for the demo.

5. Build and deploy. I ran compile steps 7 to 9, then option 13, which produced the LiveCD ISO (XigmaNAS-x64-LiveCD-14.4.0.5.10655.iso). Getting it to the PC took some network debugging (the VMware subnet mismatch and sshd config). I installed it into a second VM.

6. Post-install debugging. The WebGUI returned HTTP 500. I traced it with php -l and php-cgi with errors on:

- Missing apcu.so (the failed APCu port), fixed by renaming apcu.ini.
- A call to array_last(), which needs PHP 8.5 while the image ships 8.4. I fixed it with a small polyfill loaded through auto_prepend_file.

7. Result. The new VM shows the “Ridger NAS” login logo and the “R” favicon, and the System Information page shows my own compile date. These are the visible differences from the original. The two config strings are in the image, but the welcome text hasn’t appeared anywhere in the GUI yet.

Junway Lin

Question 3

3.1. Present relevant code, release packages, apps, stacks, and other achievements that you have already developed during your study, internship or work;

Selecting my most relevant repo: https://github.com/linjunway/Embedded-Systems-Sorting-Machine/blob/main/Code/Modules/control/control.ino

All other repos utilize python while this uses C++, which is closest to the requested language C.

This repo contained the project design, files, and source code for my final year project to build an embedded system using custom controller hardware and software design.

For the purpose of this task, I will focus more on the software side.

3.2. Explain their principles, architecture and development process;

How it works

- **Hardware:** a custom designed circuit board, powered by ATmega328 running Arduino C++ drives everything.
    - Two TCS34725 colour sensors share one I2C address, so a TCA9548A multiplexer (tcaselect()) switches between them.
    - Three IR sensors detect a ball in the 40 mm lane, a ball in the 38 mm lane, and activity at the exit.
    - Two servos act as gates and two solenoids push the ball. A motor on the AIN1/AIN2/PWM pins runs the feed, and a button starts it.
- **Sorting flow:** leverages a two phase sorting, first leveraging mechanical sorting for its speed and accuracy, then passes to the second phase where an IR sensor sees a ball and the code waits 200 ms for it to settle. It then reads the colour sensor (one stale frame discarded, then 3 samples averaged) and computes the R/G ratio. Above the orange threshold it counts as orange, below the white threshold as white, and anything between is “unknown” and is left alone. The servo gate moves, then the solenoid fires.
- **Safety and robustness:**
    - A clear-channel check rejects readings with too little or too much light.

- A 1200 ms solenoid cooldown protects the coil since too low of a cooldown period resulted in the MOSFET overheating and risked a short circuit. However, it was equally important to balance latency of the system. Hence, the cooldown accounts for the time when it isn't firing
- The motor stops after 10 s with no activity.

3.3. Modify part of the code, and introduce and explain the differences after modification;

```
unsigned long count40Orange = 0, count40White = 0;
unsigned long count38Orange = 0, count38White = 0;

// then under process40mmball:
if (fireSolenoid(SOLENOID1, 1)) {
        if (isOrange) count40Orange++; else count40White++;
        printCounts();
  }

// under process38mmball, use the corresponding variables

// finally add the new function
void printCounts() {
        Serial.print("40mm orange/white: ");
        Serial.print(count40Orange); Serial.print(" / "); Serial.println(count40White);
        Serial.print("38mm orange/white: ");
        Serial.print(count38Orange); Serial.print(" / "); Serial.println(count38White);
  }
```

Junway Lin

What Changed?

Before: the Serial output only shows per-ball readings.

After: each successful sort also prints running totals.

Fallback (one line): change MIN_RG_40 from 1.7 to 1.5. A ball with R/G around 1.6 goes from “unknown” to ORANGE, which shows how the dead band works. It’s quick, but it’s a tuning change rather than a code change, so I’d lead with the counters.

However, since I no longer have access to the hardware that runs the program, I am unable to directly test the changes (workaround by simulating to just verify no syntax errors and that it compiles).